

1. DIP – dependency inversion principle :

: Dependency Inversion Principle (DIP)

- تعریف DIP : یکی از اصول SOLID در طراحی شیء‌گرا است که بیان می‌کند:
 - ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند. هر دو باید به انتزاع interface یا abstract class وابسته باشند.
 - انتزاع نباید به جزئیات وابسته باشد؛ بلکه جزئیات باید به انتزاع وابسته باشند.
- هدف : کاهش وابستگی مستقیم بین ماژول‌ها برای افزایش انعطاف‌پذیری، قابلیت تست، و مقیاس‌پذیری کد.

: Inversion of Control (IoC)

- تعریف : مفهومی در طراحی نرم‌افزار است که کنترل جریان وابستگی‌ها (dependencies) را از کد اصلی برنامه (business logic) به یک چارچوب یا ابزار خارجی مثل IoC Container منتقل می‌کند.
- هدف : جداسازی وظایف و کاهش coupling بین اجزای برنامه.
- مثال : به جای اینکه یک کلاس مستقیماً اشیاء مورد نیاز خود را بسازد، آن‌ها را از یک منبع خارجی دریافت می‌کند.

: IoC Container

- تعریف IoC Container : یک چارچوب یا کتابخانه است که مسئول مدیریت وابستگی‌ها (dependencies) و تزریق آن‌ها (Dependency Injection) به کلاس‌ها است.
- وظایف IoC Container
 - Register: ثبت وابستگی‌ها و نوع پیاده‌سازی آن‌ها.
 - Resolve: فراهم کردن یا ساخت اشیاء مورد نیاز.
 - Lifecycle Management: مدیریت چرخه عمر اشیاء. (Singleton, Scoped, Transient)
- مثال در .NET: ابزارهایی مانند Microsoft.Extensions.DependencyInjection

: Factory as IoC Container

- تعریف : الگوی طراحی Factory یک الگوی creational است که وظیفه ساخت و بازگرداندن اشیاء را به عهده دارد. وقتی از Factory برای مدیریت وابستگی‌ها و تزریق آن‌ها استفاده می‌کنیم، می‌توان آن را به عنوان IoC Container ساده در نظر گرفت.
- تفاوت با IoC Container
 - Factory یک راه‌حل ساده برای ساخت اشیاء است و به اندازه IoC Container امکانات پیشرفته مانند مدیریت چرخه عمر، auto-wiring و غیره را ندارد.
 - IoC Container پیچیده‌تر و انعطاف‌پذیرتر است و وابستگی‌ها را به صورت پویا مدیریت می‌کند.

: Dependency Injection (DI)

- تعریف : یک الگوی طراحی است که وابستگی‌های مورد نیاز یک کلاس را به جای اینکه خود کلاس ایجاد کند، از بیرون external source دریافت می‌کند. این وابستگی‌ها معمولاً توسط یک IoC Container مدیریت و تزریق می‌شوند.
- مزایا:
 - کاهش coupling
 - افزایش قابلیت تست (testability)
 - تسهیل تغییرات در کد مثلاً جایگزینی پیاده‌سازی‌های مختلف یک interface

• انواع: Dependency Injection

- Constructor Injection : تزریق وابستگی‌ها از طریق سازنده (constructor)
- Property Injection : تزریق وابستگی‌ها از طریق ویژگی‌ها (properties)
- Method Injection : تزریق وابستگی‌ها از طریق متدها.

: Service Lifetimes in .NET

- تعریف : چرخه عمر (Lifetime) یک سرویس مشخص می‌کند که یک نمونه از سرویس چقدر زنده بماند و چگونه مدیریت شود.
- انواع در .NET:
 - Singleton: یک شیء واحد برای کل طول عمر برنامه ایجاد می‌شود.
 - Scoped: یک شیء جدید برای هر درخواست (request) ایجاد می‌شود و تا پایان آن درخواست معتبر است.
 - Transient: یک شیء جدید هر بار که درخواست می‌شود ایجاد می‌شود.

ارتباط مفاهیم :

1. DIP و IoC

- DIP اصل طراحی است که توصیه می‌کند به انتزاع وابسته باشید، نه به جزئیات IoC. روشی است که به تحقق DIP کمک می‌کند، زیرا کنترل وابستگی‌ها را از ماژول‌های سطح بالا به یک ابزار خارجی مثل IoC Container می‌دهد.

2. IoC و IoC Container

- IoC یک مفهوم کلی است، اما IoC Container ابزاری خاص برای پیاده‌سازی آن است. IoC Container وظایف مانند ایجاد و مدیریت وابستگی‌ها را به صورت خودکار انجام می‌دهد.

3. IoC Container و Factory

- Factory یک الگوی ساده‌تر برای مدیریت ایجاد اشیاء است، اما IoC Container علاوه بر این، تزریق وابستگی‌ها و مدیریت چرخه عمر آن‌ها را نیز انجام می‌دهد.

4. DI و IoC Container

- DI یکی از روش‌های پیاده‌سازی IoC است. IoC Container معمولاً ابزار اصلی برای اجرای DI است، چرا که وابستگی‌ها را تزریق می‌کند.

5. IoC Container و Service Lifetimes

- IoC Container در .NET از چرخه‌های عمر سرویس‌ها (Service Lifetimes) برای مدیریت زمان ساخت و تخریب اشیاء استفاده می‌کند.